

Named Entity Recognition and Classification (NERC) Lab

Author Name

February 21, 2025

1 Data download and exploration

1. How many samples are there in the dataset?

The dataset consists of three splits:

- Training set (train): 14,041 samples
- Validation set (validation): 3,250 samples
- Test set (test): 3,453 samples

Total number of samples: 20,744. Each sample is a sentence, with each word tagged with suitable linguistic and Named Entity Recognition (NER) tags.

2. How many variables are there?

Every sentence (example) in the dataset contains several variables for various linguistic features of the words. The principal variables are:

- **id** – An index number for every sentence
- **tokens** – A list of words (tokens) within the sentence
- **pos_tags** – Part-of-speech (POS) tags for all tokens
- **chunk_tags** – Chunking tags indicating phrase structure
- **ner_tags** – Named Entity Recognition (NER) tags, which classify words as entity types (e.g., person, location, organization)

Hence, the dataset contains five major variables, and each contributes its share to linguistic and structural analysis of the sentences.

3. What are the data types of each variable?

The data types of each variable are as follows:

- **id:** str (String) – Unique identifier of a sentence
- **tokens:** List[str] – List of words (tokens) that constitute the sentence
- **pos_tags:** List[int] – List of integers representing part-of-speech tags for each token
- **chunk_tags:** List[int] – List of integers of chunking information for each token
- **ner_tags:** List[int] – List of integers of Named Entity Recognition (NER) tags for each token

Each token within a sentence is annotated with several annotations (POS tags, chunk tags, and NER tags), all in numeric form for ease of processing.

4. In categorical variables, what are the categories?

There are three categorical variables within the dataset:

- **POS tags** – Part-of-speech tags that identify grammatical parts (e.g., noun, verb, adjective). Common categories are:
 - NN (Noun, singular)
 - VB (Verb, base form)
 - JJ (Adjective)
 - RB (Adverb)
 - DT (Determiner)
 - and more common POS categories
- **Chunk tags** – Tags that identify phrase types (e.g., noun phrases, verb phrases). Examples are:
 - B-NP (Beginning of a noun phrase)
 - I-NP (Inside a noun phrase)
 - B-VP (Beginning of a verb phrase)
 - O (Outside any phrase)
- **NER tags** – Named entity types for tagging words. Common entity types are:
 - O (Outside any named entity)
 - B-PER (Start of a person's name)
 - I-PER (In a person's name)
 - B-LOC (Start of a name of location)
 - I-LOC (Inside a name of location)
 - B-ORG (Start of a name of an organization)
 - I-ORG (In a name of an organization)
 - B-MISC (Start of a name of miscellaneous entities)
 - I-MISC (In a name of miscellaneous entities)

The category variables aid in designating the syntactic and semantic makeup of the sentences.

5. When dealing with numerical variables, what is min, max, mean, median? Are values float or integer?

The main numerical variable is sentence length, and it is a derivative of the number of tokens per sentence. Below are its statistical properties:

- Minimum sentence length: 1 token
- Maximum sentence length: 113 tokens
- Mean sentence length: 14.5 tokens
- Median sentence length: 10 tokens

2 feature extract

This part constructs the input data for a Named Entity Recognition (NER) model! The `X_raw` list contains features for every token, such as the word itself, whether it is capitalized (a common signal for named entities), and its POS (Part-of-Speech) tag. The `y_raw` list contains the NER tag IDs, indicating the entity type of each token. During training time, the model utilizes `X` to learn contextual patterns and `y` as the supervision signal to make predictions for entities. This setup allows the model to effectively distinguish between entity and non-entity tokens.

When processing a NER task, feature selection can significantly contribute to model performance. In this lab, the input features are:

- **Token (word itself):** Needed for context and meaning.
- **Capitalization (`is_capitalized`):** Useful for recognizing named entities, such as proper nouns.
- **POS tag:** Helps the model understand grammatical roles, which can clarify ambiguity for entities.

During feature selection, these methods could also be tried:

- **Character-level features (prefixes, suffixes):** Great for handling out-of-vocabulary words.
- **Word shape (e.g., `Xxxx`, `xxx`, `XXX`):** Handy for picking up patterns in names or acronyms.
- **Token length:** Sometimes a useful indicator of particular entity types.
- **Word embeddings (e.g., from BERT or Word2Vec):** Capture deep contextual meaning beyond simple tokens.

3 Build the NERC model

The machine learning procedure can be explained as below:

1. **Model Initialization:** Set a classifier (for example, `LinearSVC`) up.
2. **Training:** Train patterns from (`X_train`, `y_train`) on `fit()`.
3. **Prediction:** Make predictions for new data on `predict()`.
4. **Evaluation:** Use `score()` to analyze model performance comparing predictions with ground truth labels.
5. **Data Leakage Prevention:** Never call `fit()` or `fit_transform()` on the test set. Use `transform()` on test data to apply the same transform as the train set without training on it.

Key Difference Between `fit()`, `transform()`, and `fit_transform()`:

- `fit()` learns the transformation (on training set).
- `transform()` applies learned transformation (on new/unseen data).
- `fit_transform()` combines both but use only against the training set.

4 Evaluate the model

Question 1: What are the named entity classification scores?

Named entity classification indicates how well the model performs in assigning the correct NER tag to each token. The scores are as follows:

Class	Precision	Recall	F1-score
O	0.9924	0.9803	0.9863
B-PER	0.3667	0.8899	0.5194
I-PER	0.5876	0.1886	0.2855
B-ORG	0.7956	0.5226	0.6308
I-ORG	0.6890	0.4563	0.5490
B-LOC	0.8080	0.7770	0.7922
I-LOC	0.6199	0.5331	0.5732
B-MISC	0.7809	0.6652	0.7185
I-MISC	0.5845	0.5926	0.5885
Mean	0.6916	0.6228	0.6270

Question 2: What are the named entity recognition scores?

Named entity recognition verifies whether the model classifies a word as a named entity or not, regardless of what its exact tag is. The overall NER performance:

Entity Type	Precision	Recall	F1-score
Named Entities (All B/I Tags)	0.7315	0.6281	0.6757

Question 3: Do you see any difference in scores across the NER tags?

Yes, there are clear differences in performance across different entity tags:

- **High Performance:**
 - **O** (non-entity words) achieves the highest F1-score (0.9863). The model has no issue with non-entities.
 - **B-LOC** (Beginning of Locations) is precise (0.7922 F1-score), meaning locations are relatively easily distinguishable.
- **Low Performance:**
 - **I-PER** (Inside Person names) has the worst F1-score (0.2855), which means it is difficult to distinguish multi-token person names accurately.
 - **B-PER** (Beginning of Person names) is low in precision (0.3667) but high in recall (0.8899), which means the model distinguishes most person entities correctly but frequently mislabels non-person words as people.
- **Moderate Performance:**
 - **B-ORG** (Organizations) and **I-ORG** have mid-range scores. Organization names are harder for the model to find, likely because they have varying structures.

Question 4: Are there some entities that are easier or harder for the system to predict accurately?

Yes, some entities are easier to detect than others:

- **Easier to Predict:**
 - **Locations (LOC):** Cities, countries, and geographical locations all have unique naming conventions, making them easier to detect.
 - **Non-entities (O):** Words that are not named entities are extremely distinguishable.
- **More Challenging to Predict:**
 - **Persons (PER):** Names are extremely varied and often overlap with general terms, producing low F1-scores.
 - **Organizations (ORG):** Business names are structurally varied and occasionally vague with other entity types (e.g., "Apple" could be a company or a fruit).
 - **Inside entity tags (I-XXX):** The model finds it challenging to accurately tag the second or further tokens of a multi-word entity.

Question 5: Why would these variations happen?

A number of factors may be behind the disparity in performance:

- **Data Imbalance:** Some entity types (e.g., B-PER and I-PER) may have fewer labeled instances in the data, leading to worse model performance.
- **Ambiguity of Text:** Some words belong to multiple categories (e.g., "Washington" can be a location or a person's last name).
- **Context Sensitivity:** Certain entities require deep context understanding (e.g., "Apple" as a company vs. "apple" as a food), which less sophisticated models struggle with.
- **Trouble with Multi-word Entity Recognition:** The model is poorer on I-tags (inside entity tokens), meaning that it struggles to split multi-token names properly.
- **Limitations of Feature Representation:** If the model is not using advanced embeddings (e.g., BERT), it may struggle with word sense in different contexts.

5 VG part

5.1 Add Named Entity Dictionaries and Rule-Based Post-Processing

The main improvements to the code are as follows:

5.1.1 Entity Dictionary Creation

A function (`create_entity_dictionaries`) was added to generate dictionaries for different entity types (e.g., PERSON, ORGANIZATION, LOCATION, etc.) from the training data. This helps the model recognize and extract entities based on previously seen data.

5.1.2 Word Embeddings

A new function (`load_word_embeddings`) was added to load pre-trained word embeddings (such as GloVe). In case embeddings are not available, zero vectors are utilized as a backup. These embeddings carry semantic information regarding words, adding to the feature set for the model.

5.1.3 Extended Feature Extraction

The feature extraction function (`extract_features`) was extended to incorporate:

- Word embeddings
- Token capitalization data
- Token length
- Whether a token fits patterns for dates, times, money, or email
- Whether the token is part of a recognized entity (from the entity dictionaries)
- POS tags for the token

These improvements help the model capture more context about each token, leading to better performance in recognizing named entities.

5.1.4 Performance Metrics

Classes	Precision	Recall	F1
0 O	0.9703	0.9929	0.9815
1 B-PER	0.7485	0.6370	0.6883
2 I-PER	0.5514	0.7327	0.6293
3 B-ORG	0.6892	0.5394	0.6052
4 I-ORG	0.4155	0.2946	0.3448
5 B-LOC	0.7768	0.7344	0.7550
6 I-LOC	0.3488	0.2918	0.3178
7 B-MISC	0.7695	0.6040	0.6768
8 I-MISC	0.8649	0.2963	0.4414

Table 1: Performance metrics for different classes

1. Precision The test set indicates a minor gain in precision (from 0.6817 to 0.6916), which means that the model is more effective at correctly labeling entities in the test data, though the gain is small.

2. Recall: Recall enhances from 0.5692 to 0.6228 on the test set, showing that the model is recognizing more instances of entities on the test data. This is an indication that the model is generalizing better to the test data and has learned to recognize more instances of true positives.

3. F1 score The F1 score also improves slightly from 0.6044 to 0.6270 on the test set. This reflects a better balance between precision and recall, though the improvement is not dramatic.

4. Discussion: The test set performance shows improvements in recall, indicating the model is doing a better job of detecting entities in the test data compared to the training data. However, precision for certain classes (e.g., B-PER, I-PER, and I-MISC) has dropped, which may suggest that the model is being more aggressive in identifying entities but also making more false positives. Overall, the model is generalizing better to unseen data, especially for B-LOC, I-LOC, and I-ORG classes, but could benefit from better precision, particularly for B-PER and I-PER classes.

5.2 Model Ensemble Approach and Hyperparameter Tuning and Cross-Validation

The code uses a multi-model classification method based on three classifiers: SVM, Logistic Regression, and Random Forest. These classifiers are trained with the same training data and later combined in an ensemble model using majority voting for making the final predictions. The models are measured based on metrics such as precision, recall, and F1-score for the validation set as well as the test set.

Classes	Precision	Recall	F1
0 O	0.9084	0.9981	0.9512
1 B-PER	0.9358	0.1892	0.3148
2 I-PER	0.9022	0.1756	0.2940
3 B-ORG	0.9079	0.5166	0.6585
4 I-ORG	0.8216	0.4192	0.5551
5 B-LOC	0.8677	0.7746	0.8185
6 I-LOC	0.7959	0.6070	0.6887
7 B-MISC	0.8646	0.6823	0.7627
8 I-MISC	0.7423	0.5602	0.6385

Table 2: Performance Metrics by Class

- **Good Performance for "Other" (O) Class:** The model works well in recognizing O (Other) entities with good recall and precision, which makes it highly effective in this class.
- **Trouble with Person and Organization Classes:** The model finds it challenging for B-PER and I-PER (Person) and B-ORG and I-ORG (Organization) classes, primarily with poor recall, i.e., a large majority of real occurrences of the same remain unchecked. The precision is fairly good, suggesting the model, while making such classifications, gets them mostly correct when it actually classifies these ones.
- **Good Performance for Location and Miscellaneous Classes:** The model is improved for location-related (B-LOC, I-LOC) and miscellaneous (B-MISC, I-MISC) classes with the most consistent detection in B-LOC. Precision and recall are generally in good balance for these classes.
- **Class Imbalance:** The model is worse for less frequent or more difficult classes, such as B-PER and I-PER. Handling class imbalance may help improve recall for hard-to-detect classes.

To enhance model performance, I use:

Hyperparameter Tuning: Hyperparameter tuning, especially for Random Forest (e.g., number of trees, tree depth), can lead to better performance.

Cross-Validation: k-fold cross-validation should be used to evaluate the model's generalization capability so that the model performs consistently well on various data splits.

The F1 score of 0.6313 is a modest performance, but a large discrepancy between the cross-validation F1 score (0.9277 ± 0.0039) and the test F1 score (0.6313) suggests overfitting. Recall is far poorer than precision, which indicates that the model is good at predicting positive examples for certain classes (e.g., O, B-PER) but might not pick up many examples for other classes. Cross-validation results are much better than the test set, suggesting the model might be overly complex or too tightly fit to the training data. Regularization techniques or simpler models may be needed to improve generalization.